

pdf2json — Complete Documentation

What does this program do?

It takes **one PDF file** and converts everything inside it into **one structured JSON file**. Every heading, paragraph, table, circuit diagram, and photo becomes a numbered block in the JSON.

Input: any PDF — reports, datasheets, schematics, manuals (up to 20,000 pages)

Output 1: a `.json` file with every element as a structured block

Output 2: an `extracted_images/` folder with every diagram and photo saved as PNG

How to run it

Simplest command

```
python pdf2json.py "C:\Users\divin\Downloads\myfile.pdf"
```

Produces `myfile.json` in the same folder as your PDF.

With diagram extraction

```
python pdf2json.py "C:\path\to\myfile.pdf" --vector-crop
```

Without `--vector-crop`, only embedded photos are extracted.

With it, circuit diagrams and block diagrams are also detected and saved as PNG.

Full example

```
python pdf2json.py "C:\path\to\myfile.pdf" ^
```

```
--output "C:\output\result.json" ^
```

```
--images-dir "C:\output\images" ^
```

```
--vector-crop ^
```

--workers 4 ^

--log-level DEBUG

For very large documents (streaming)

python pdf2json.py "C:\path\to\bigfile.pdf" ^

--stream ^

--chunk-size 200 ^

--output-dir "C:\output\chunks" ^

--workers 8

Instead of one big JSON, writes one file per 200 pages into the chunks folder. Use this for thousands of pages to avoid running out of RAM.

All available options

Option	Default	What it does
--output	next to PDF	Where to save the JSON file
--images-dir	./extracted_images	Where to save extracted PNG images
--vector-crop	off	Enable circuit/schematic/diagram detection
--workers N	4	Number of parallel threads
--page-start N	1	First page to process
--page-end N	last	Last page to process
--table-method	auto	Force a strategy: auto / lattice / stream / camelot
--no-tables	off	Skip table extraction entirely
--no-images	off	Skip image extraction entirely

Option	Default	What it does
<code>--dpi N</code>	150	Resolution for saved diagram PNGs
<code>--stream</code>	off	Write JSON in chunks (for large documents)
<code>--chunk-size N</code>	500	Pages per chunk in streaming mode
<code>--log-level</code>	INFO	Verbosity: DEBUG / INFO / WARNING / ERROR

How it works — step by step

For every page in the PDF, the program runs this exact sequence. **The order matters.**

Analyze page → Extract images → Extract tables → Extract text → Merge & sort

Step 1 — Analyze the page (`page_analyzer.py`)

Before running any extractor, the program does a cheap scan (milliseconds per page).

It checks:

- Does this page have text? (`get_text("words")`)
- Does it have horizontal and vertical lines suggesting a table? (`get_drawings()`)
- Does it have embedded photos? (`get_images()`)
- Does it have vector drawing paths (lines, rectangles, arcs)?

The answers are stored in a `PageProfile` and control which extractors run next.

A page with no drawings skips the diagram extractor entirely — this makes the program fast on large documents.

Step 2 — Extract images first (`image_extractor.py`)

Images are extracted **before text on purpose**. Their bounding boxes become *exclusion zones* — areas the text extractor must skip. Without this, every wire label inside a circuit diagram (`A`, `B`, `GND`) would appear as a stray paragraph in the JSON.

Track A — raster images (photos, embedded PNGs):

- PyMuPDF finds each image by its internal reference number (xref)
- Decodes the raw bytes into pixels, converts CMYK to RGB if needed
- Saves as PNG, records position on the page

Track B — vector diagram regions (circuits, schematics, block diagrams):

- `get_drawings()` returns every path on the page: lines, rectangles, arcs
- Zero-area paths (pure lines) get a 1×1 point footprint so they participate in grouping
- Nearby paths are grouped together (any two paths within 80 points are in the same group)
- Groups that are large enough (>2000 pt²) and complex enough (>3 paths) and not just table borders are rendered as PNG using `get_pixmap(clip=region)`

Step 3 — Extract tables (`table_extractor.py`)

Three strategies are tried in order, stopping at the first success:

1. **Lattice** (pdfplumber): finds grid lines and uses their intersections as cell borders. Works for tables with visible borders. Accuracy: 90%.
2. **Stream** (pdfplumber): infers columns from word spacing. Works for borderless tables. Accuracy: 75%.
3. **Camelot** (fallback): renders the page as a bitmap and detects lines by image analysis. Slowest but most powerful.

The strategy used is recorded in the output as `extraction_method`.

Step 4 — Extract text, avoiding image/table zones (`text_extractor.py`)

PyMuPDF's `get_text("dict")` returns a tree: blocks → lines → spans. Each span is a run of characters sharing the same font and size.

The extractor:

1. Computes the **median font size** across all spans on the page (= body text size)
2. Classifies blocks larger than 115% of body size (or bold and short) as **headings**
 - ×2.0 bigger = level 1, ×1.5 = level 2, ×1.2 = level 3, otherwise level 4
3. Classifies everything else as **paragraphs**
4. Drops any block whose bounding box overlaps more than 25% with an image or table zone

Step 5 — Merge and sort (`element_merger.py`)

All elements from all three extractors are combined and cleaned:

1. **Caption deduplication:** caption text already stored inside an image block is dropped so it doesn't appear twice as a paragraph
 2. **Expanded exclusion zones:** vector diagram regions are expanded by 45 points in all directions to suppress wire labels sitting just outside the drawn boundary
 3. **Final overlap filter:** text elements overlapping exclusion zones by more than 30% are dropped
 4. **Reading order sort:** everything sorted by `(y0, x0)` — top of page first, then left to right
-

File-by-file reference

`pdf2json.py` — entry point

The only file you run. Parses command-line arguments, sets up logging, calls the pipeline, writes the output. The logger is built directly into this file — no separate `logger.py` needed.

`pipeline.py` — the orchestrator

Opens the PDF twice simultaneously — once with PyMuPDF (fast, gives access to drawing paths and raw image data) and once with pdfplumber (specialised for table line detection). Loops over every page, calls `page_analyzer` first, then runs extractors in the correct order.

In parallel mode (`--workers N`), each worker thread opens its own copy of the PDF because PyMuPDF is not thread-safe with shared file handles.

`page_analyzer.py` — the gatekeeper

Runs in milliseconds per page. Returns a `PageProfile` dataclass with booleans. Pages that return `has_tables=False` skip the table extractor entirely — this is the main performance optimization.

`text_extractor.py` — reading text

Calls `page.get_text("dict")`. Bold is detected by bit 4 of the font flags (value 16). Italic is bit 1 (value 2). The median font size across all spans on the page is used as the baseline for heading detection.

`table_extractor.py` — reading tables

Each cell goes through `_clean_cell()` which collapses internal newlines to spaces — important for multi-line cells. The first non-empty row is always treated as the header row.

image_extractor.py — reading images

Track A: `fitz.Pixmap(doc, xref)` decodes raw bytes. CMYK images are converted to RGB. Images under 1000 pt² are skipped (icons, bullets).

Track B: zero-area paths get a 1×1 point footprint. Clusters are built with union-find. Surviving clusters are rendered with `page.get_pixmap(clip=rect)`. Table borders are excluded by checking that the cluster contains at least 2 box-shaped (2D) paths.

element_merger.py — cleaning up

Three operations: (1) collect captions already in image blocks, drop duplicate text elements. (2) Build exclusion zones — exact bboxes for tables and raster images, but expanded by 45pt for vector regions. (3) Drop text overlapping zones by more than 30%. (4) Sort by `(y0, x0)`.

bbox_utils.py — geometry

All bounding boxes are `(x0, y0, x1, y1)` tuples in PDF points, origin at top-left. Key functions: `area()`, `intersection()`, `iou()` (intersection-over-union), `overlap_ratio()`, `cluster_bboxes()` (union-find grouping), `merge_cluster_to_bbox()`.

caption_finder.py — finding captions

For each image bbox, scans all text blocks on the page. A block qualifies if it horizontally overlaps the image by at least 25% and sits within 48pt below or 24pt above. Blocks starting with *Figure*, *Fig.*, *Table*, *Diagram*, *Schematic* etc. are preferred.

schema_v1.py — writing JSON

Uses `orjson` if installed (10–20× faster than `stdlib json`), falls back to `json.dump` transparently. In streaming mode, writes one numbered chunk file per batch of pages.

id_generator.py — unique IDs

Every element gets an ID like `elem_000042_p0007` (42nd element overall, found on page 7). Thread-safe counter using a `threading.Lock`.

JSON output format

Document wrapper

```
{
```

```
"document_metadata": {  
  
  "title": "Equipment Specification Language",  
  
  "author": "J. Martin",  
  
  "page_count": 45,  
  
  "source_pdf": "myfile.pdf",  
  
  "extraction_date": "2026-05-27T10:00:00Z"  
  
},  
  
"stats": {  
  
  "total_elements": 312,  
  
  "paragraphs": 180,  
  
  "headings": 42,  
  
  "tables": 18,  
  
  "images": 72,  
  
  "elapsed_seconds": 8.4  
  
},  
  
"content": [ ...all blocks... ]  
  
}
```

Heading block

```
{  
  
  "id":      "elem_000001_p0001",  
  
  "type":    "heading",  
  
  "level":   2,
```

```
"page":    1,

"bounding_box": [72.0, 52.0, 480.0, 78.0],

"text":    "1. System Overview",

"font":    "Helvetica-Bold",

"font_size": 14.0

}
```

Paragraph block

```
{

  "id":      "elem_000002_p0001",

  "type":    "paragraph",

  "page":    1,

  "bounding_box": [72.0, 90.0, 520.0, 145.0],

  "text":    "The system shall transition to alert state within 50ms.",

  "font":    "Helvetica",

  "font_size": 10.0,

  "bold":    false,

  "italic":  false

}
```

Table block

```
{

  "id":      "elem_000003_p0001",

  "type":    "table",
```



```
"page": 1,

"bounding_box": [72.0, 160.0, 520.0, 280.0],

"extraction_method": "lattice",

"accuracy": 90.0,

"col_count": 4,

"row_count": 2,

"headers": ["Signal Name", "Type", "Range", "Description"],

"rows": [

    ["In_Engine_Fire_L", "Boolean", "N/A", "Left engine fire detection."],

    ["In_Engine_Fire_R", "Boolean", "N/A", "Right engine fire detection."]

]

}
```

Raster image block

```
{

    "id": "elem_000004_p0003",

    "type": "image",

    "subtype": "raster",

    "page": 3,

    "bounding_box": [100.0, 150.0, 450.0, 300.0],

    "extracted_image_path": "C:\\output\\images\\img_p0003_001.png",

    "width_px": 800,
```

```
"height_px":      600,

"colorspace":     "DeviceRGB",

"caption":        "Figure 2.1: Pressure sensor assembly."

}
```

Vector diagram block

```
{

  "id":            "elem_000005_p0004",

  "type":          "image",

  "subtype":       "vector_region",

  "page":          4,

  "bounding_box":  [80.0, 110.0, 500.0, 420.0],

  "extracted_image_path": "C:\\output\\images\\vec_p0004_001.png",

  "width_px":      992,

  "height_px":     686,

  "path_count":    47,

  "caption":       "Figure 3.2: Functional block diagram."

}
```

`path_count` tells you how many drawing paths were in the cluster. A simple AND gate might have 8. A full system schematic might have 300.

Tuning for your document type

If the default output is not right for your specific PDFs, edit these constants.

In `image_extractor.py`

Constant	Default	When to change
<code>MIN_VECTOR_AREA</code>	2000 pt ²	Increase to skip small decorative shapes. Decrease for tiny logic gate symbols.
<code>MIN_PATH_COUNT</code>	3	Increase to require more complex diagrams.
<code>VECTOR_CLUSTER_GAP</code>	80 pt	Increase if block diagram components are splitting into multiple images. Decrease if two separate diagrams are merging.
<code>VECTOR_DPI</code>	150	Increase for sharper exports (300 = print quality).
<code>MIN_IMAGE_AREA</code>	1000 pt ²	Increase to skip tiny icons and decorative elements.

In `element_merger.py`

Constant	Default	When to change
<code>TEXT_INSIDE_THRESHOLD</code>	0.30	Lower if diagram labels still appear as paragraphs. Raise if real text near diagrams is incorrectly suppressed.
<code>DIAGRAM_LABEL_MARGIN</code>	45 pt	Increase if short wire labels (A, B, GND) still appear as paragraphs. Decrease if real paragraphs near diagrams are suppressed.

In `page_analyzer.py`

Constant	Default	When to change
<code>MIN_LINE_LENGTH</code>	30 pt	Increase if decorative short lines trigger false table detection.

Known limitations

Scanned PDFs: If the PDF was created by scanning paper without OCR, there is no text layer. All content appears as raster images — no text or tables are extracted. You would need to add an OCR step (e.g. pytesseract) first. How to tell: if every page only produces image blocks with no paragraphs, it is a scanned PDF.

Multi-column layout: The reading order sort uses `(y0, x0)`. In a two-column document, elements may appear in the wrong order in the JSON if a right-column element starts higher on the page than a left-column element.

Overlapping diagrams: If two unrelated diagrams sit within 80 points of each other on the same page, they merge into one image. Reduce `VECTOR_CLUSTER_GAP` to prevent this.

Mathematical formulas: Inline equations are captured as text but the character order may be wrong for complex encodings. Large display equations are captured as vector diagram images.

Rotated pages: Text extraction works (PyMuPDF normalises rotation). Tables on rotated pages may confuse pdfplumber — use `--table-method camelot` as a workaround.